

pathlib in Python

Object-oriented, cross-platform filesystem paths

Before `pathlib`, working with file paths in Python meant string manipulation with `os.path` — joining paths with `os.path.join()`, checking existence with `os.path.exists()`, and so on. `pathlib` (standard library since Python 3.4) replaces this with a **Path object** that represents a filesystem path and carries its own methods for reading, writing, joining, and inspecting — and it handles Windows vs. Unix path separators automatically.

os.path (old style)	pathlib (modern style)
<code>os.path.join(a, b)</code>	<code>Path(a) / b</code>
<code>os.path.exists(p)</code>	<code>Path(p).exists()</code>
<code>os.path.isfile(p)</code>	<code>Path(p).is_file()</code>
<code>os.path.splitext(p)</code>	<code>Path(p).suffix</code>
<code>open(p).read()</code>	<code>Path(p).read_text()</code>
<code>os.makedirs(p)</code>	<code>Path(p).mkdir(parents=True)</code>

1. Creating Path Objects

1.1 Basic construction

```
from pathlib import Path

p = Path("data/raw/file.csv") # relative path
p2 = Path("/home/user/data") # absolute path (Unix-style, works cross-platform)
p3 = Path.cwd() # current working directory
p4 = Path.home() # user's home directory
```

1.2 Joining paths with the / operator

This is the signature pathlib feature: use `/` instead of string concatenation or `os.path.join()`. It automatically uses the correct separator for the operating system (`/` on Unix, `\\` on Windows).

```
base = Path("data")
file_path = base / "raw" / "2026" / "sales.csv"
print(file_path) # data/raw/2026/sales.csv (or data\\raw\\2026\\sales.csv on Windows)
```

■ Cross-platform by design

Code written with `pathlib` works identically on Windows, macOS, and Linux without any manual separator handling — a common source of bugs with plain string paths.

2. Inspecting Paths

Path objects expose the pieces of a path as properties, without any string splitting:

```
p = Path("/home/user/data/sales_2026.csv")

p.name # 'sales_2026.csv' - final component
p.stem # 'sales_2026' - name without suffix
p.suffix # '.csv' - file extension
p.suffixes # ['.csv'] - all extensions (e.g. ['.tar', '.gz'])
p.parent # Path('/home/user/data') - containing directory
p.parents[0] # same as p.parent
p.parents[1] # Path('/home/user') - grandparent, etc.
p.parts # ('/', 'home', 'user', 'data', 'sales_2026.csv')
p.is_absolute() # True
```

Attribute	Returns
.name	Final path component (file or folder name)
.stem	Name without the final suffix
.suffix	Final file extension, including the dot
.parent	Path object for the containing directory
.parts	Tuple of all path components
.anchor	Root/drive portion (e.g. "/" or "C:\\")

3. Checking Existence and Type

```
p = Path("data/sales.csv")

p.exists() # True if the path exists at all
p.is_file() # True if it's a regular file
p.is_dir() # True if it's a directory
p.is_symlink() # True if it's a symbolic link

stat = p.stat()
stat.st_size # file size in bytes
stat.st_mtime # last modified time (Unix timestamp)
```

■ Combine with datetime

`stat.st_mtime` is a Unix timestamp, so pair it with `datetime.fromtimestamp(stat.st_mtime, tz=timezone.utc)` to get a readable, timezone-aware modification time.

4. Reading and Writing Files

Path objects can read and write file contents directly, without a separate `open()` call for simple cases:

```
p = Path("notes.txt")

# Text
p.write_text("Hello, world!", encoding="utf-8")
content = p.read_text(encoding="utf-8")

# Bytes
p.write_bytes(b"\x00\x01\x02")
data = p.read_bytes()

# Still works as a context manager for more control
with p.open("r", encoding="utf-8") as f:
    for line in f:
        print(line)
```

■ `write_text()` overwrites, not appends

`Path.write_text()` always replaces the file's full contents. For appending, use `p.open("a")` as a context manager instead.

5. Working with Directories

5.1 Creating directories

```
Path("output").mkdir(exist_ok=True) # create one directory
Path("output/2026/09/07").mkdir(parents=True, exist_ok=True) # create nested directories
```

■ exist_ok=True and parents=True

exist_ok=True avoids a `FileExistsError` if the directory is already there. parents=True creates any missing intermediate directories — the equivalent of `mkdir -p`.

5.2 Listing contents

```
p = Path("data")

for item in p.iterdir(): # direct children only (files and folders)
    print(item)

for csv_file in p.glob("*.csv"): # pattern match, current level only
    print(csv_file)

for csv_file in p.rglob("*.csv"): # recursive - all subdirectories too
    print(csv_file)
```

Pattern	Matches
*.csv	Any file ending in .csv in this directory
.	Any file with an extension
/*.csv	.csv files in this directory and all subdirectories (with rglob or glob("/*.csv"))
data_????.json	data_ followed by exactly 4 characters, then .json

6. Renaming, Moving, and Deleting

```
p = Path("old_name.txt")

p.rename("new_name.txt") # rename or move (overwrites on some platforms)
p.replace("new_name.txt") # rename or move, always overwrites

p.unlink() # delete a file
p.unlink(missing_ok=True) # delete a file, no error if it doesn't exist

Path("empty_dir").rmdir() # delete an empty directory
```

■ No built-in recursive directory delete

`pathlib` has no method to delete a non-empty directory tree. Use `shutil.rmtree(path)` from the standard library for that.

```
import shutil
shutil.rmtree("output/old_run") # delete a directory and everything inside it
```

7. Practical Patterns

7.1 Building paths relative to a script

A common need: reference a file relative to the currently running script, regardless of the working directory it's launched from:

```
from pathlib import Path

SCRIPT_DIR = Path(__file__).resolve().parent
config_path = SCRIPT_DIR / "config" / "settings.json"
```

7.2 Changing a file's extension

```
p = Path("report.csv")
json_version = p.with_suffix(".json") # report.json
```

7.3 Changing just the filename

```
p = Path("data/raw/sales.csv")
new_path = p.with_name("sales_cleaned.csv") # data/raw/sales_cleaned.csv
```

7.4 Resolving to an absolute, normalized path

```
p = Path("../data/./sales.csv")
print(p.resolve()) # absolute path with .. and . resolved
```

7.5 Safe batch processing over a directory

```
from pathlib import Path

input_dir = Path("data/raw")
output_dir = Path("data/processed")
output_dir.mkdir(parents=True, exist_ok=True)

for csv_file in input_dir.glob("*.csv"):
    out_path = output_dir / csv_file.name
    text = csv_file.read_text(encoding="utf-8")
    out_path.write_text(text.upper(), encoding="utf-8") # example transform
```

8. Common Pitfalls

- **Forgetting Path objects aren't strings:** most string methods don't work directly on a Path — use `str(p)` to convert when a library strictly needs a string (most modern libraries accept Path objects directly).
- **glob() vs rglob():** `glob("*.csv")` only searches the immediate directory; use `rglob("*.csv")` or `glob("**/*.csv")` to search subdirectories too.
- **write_text() overwrites silently:** there's no append mode on the convenience methods — use `.open("a")` for appending.
- **No recursive delete built in:** `rmdir()` only removes empty directories; use `shutil.rmtree()` for a full tree.
- **resolve() vs absolute():** `.absolute()` doesn't resolve symlinks or `..` segments; `.resolve()` does and also normalizes the path — prefer `.resolve()` when you need a canonical path.
- **Existence race conditions:** checking `p.exists()` then acting on it isn't atomic — another process could change the filesystem in between. For critical code, handle the resulting exception instead of pre-checking.

Task	Method
Join path segments	<code>Path(a) / b / c</code>
Check existence	<code>p.exists()</code> , <code>p.is_file()</code> , <code>p.is_dir()</code>
Read/write whole file	<code>p.read_text()</code> , <code>p.write_text()</code>
Create directories	<code>p.mkdir(parents=True, exist_ok=True)</code>
List files by pattern	<code>p.glob("*.csv")</code> , <code>p.rglob("*.csv")</code>
Get absolute, normalized path	<code>p.resolve()</code>
Change extension / filename	<code>p.with_suffix()</code> , <code>p.with_name()</code>
Delete a directory tree	<code>shutil.rmtree(p)</code>